

A Trillion Vectors in the Smallest Job Class [Scalable Data Science]

Building and measuring a sharded vector index on a shared research cluster with one CPU and two gigabytes per job

Andrew H. Bond
San José State University
San José, California, USA
andrew.bond@sjsu.edu

ABSTRACT

A sharded vector index of one trillion 4-bit quantized rows, 24 TB on 500 block volumes, was built and then measured exactly on the National Research Platform, a Kubernetes cluster shared by many research groups, with no reserved capacity. Every job in the build and the measurement requested one CPU and two gigabytes of memory, the smallest class the cluster offers. Three design choices made that possible. The corpus is defined by a seed and regenerated where it is needed, so the build moved no corpus bytes and a lost volume was rebuilt from its seed in under four hours. Every job is idempotent on its volume, so a failed or duplicated job costs a retry and never a wrong result. A pool of per-server jobs, submitted through a controller that yields when the cluster is busy, recovers from failures by a small number of explicit rules rather than by operator attention. The build of 500 servers took seven weeks of background capacity, and the exact scan and five routed passes over the whole index took four days, 1349 CPU-hours, 1502 submissions and 1350 completions, with six pods replaced by hand. The recall this measured is reported in a companion paper. Here we describe the index layout, the corpus generator, the job shape, the pool and its rules, what failed and how often, and what the whole run cost.

PVLDB Reference Format:

Andrew H. Bond. A Trillion Vectors in the Smallest Job Class [Scalable Data Science]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/ahb-sjsu/turboquant-pro/tree/master/benchmarks/fleet>.

1 INTRODUCTION

Measuring a vector index at a trillion rows needs a trillion rows in one place and an exact scan of all of them. Few groups own that much capacity. Research clusters that federate many sites have it in aggregate, but they share it, reserve nothing for any one user, and expect jobs that come and go without supervision. This paper describes how a trillion-row index was built and scanned exactly

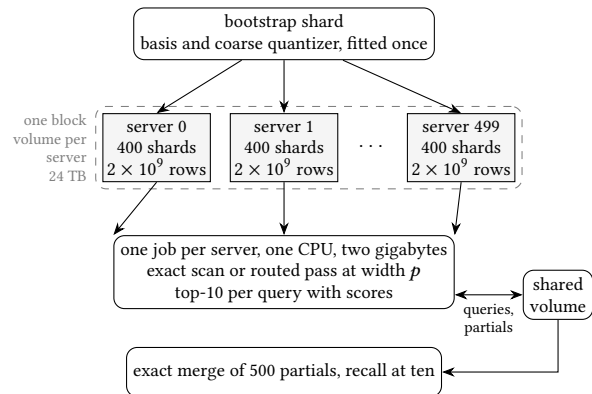


Figure 1: The fleet. One bootstrap shard fixes the projection basis and the 2048-cell coarse quantizer for every shard on every server, so scores are comparable across servers and the global top-10 is the exact merge of per-server top-10s. Each phase is one job per server, and each job reads only its own volume and the shared volume.

on such a cluster, the National Research Platform’s Nautilus [10], with every job in the smallest resource class the platform offers.

The run exists to answer a measurement question, whether routed search over a sharded index loses anything the exact scan finds as the index grows to 10^{12} rows. The companion paper [1] reports that answer. This paper is about the machinery. It makes three points. A corpus defined by a seed turns a 24 TB build into computation with no data movement and turns the loss of a volume into a rebuild. A scan over memory-mapped shards fits one CPU and two gigabytes once three choices are made about queries, open shards and block size. A pool of idempotent jobs with explicit rules for failure, waiting and deferral carries a four-day run of 1502 submissions through the ordinary failures of a shared cluster. Section 8 states what we would keep and what we would change.

2 THE INDEX IS 500 SERVERS OF TWO BILLION ROWS

Index. Each row is a 32-dimensional vector projected to 24 dimensions by one basis shared by the whole fleet, quantized to 4 bits per dimension, and assigned to one of 2048 inverted-file cells by one shared coarse quantizer. Both the basis and the quantizer are fitted once, on a bootstrap shard, and read by every build job. A shard holds five million rows. A server holds 400 shards, two

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

billion rows, on one block volume. The on-disk row is 12 bytes of codes, 8 bytes of per-row norms used by the asymmetric distance computation, and 4 bytes of cell membership. Every server reports 48 009 656 624 index bytes for its two billion rows, 24.0 bytes per row, and the fleet holds 24.0 TB of index on 500 volumes. Scoring is asymmetric distance computation over the codes, in row blocks, from memory-mapped shards. Because the basis and the coarse quantizer are shared, scores from different shards are comparable and a global top- k is the exact merge of per-shard top- k s. That property is what makes an exact measurement of the whole index possible, and Figure 1 shows the arrangement.

Scale. The same layout has been built and measured at 10^8 rows on 4 servers, 10^9 on 4, 10^{10} on 8, 10^{11} on 50 and 10^{12} on 500. Per-server work is constant from 10^{11} on, two billion rows, so the step from 50 to 500 servers changes nothing per node and changes only the number of chances to draw a slow or failing one.

3 THE CORPUS IS A SEED, NOT A FILE

The corpus is defined by a seed and regenerated on demand. Each global shard draws a fresh rank-16 Gaussian basis, projects Gaussian coefficients through it with a linear decay of singular values from 1.0 to 0.3, and adds five percent noise, in an ambient dimension of 32. A shard is a pure function of its global index, so any job can produce any shard without reading anything. The companion paper [1] discusses what this corpus is and is not as a stand-in for real embeddings. For the systems question it does three things.

It removes data movement. A 24 TB index built from files would have to move those files to 500 volumes. Built from a seed, each server’s job computes its own 400 shards and writes only the index. The shared volume holds the fitted basis, the coarse quantizer, the query cache and the partial results, which are small.

It bounds memory. A build job streams its 400 shards from the seed in row bands, writes each shard as it goes, and never holds a whole shard in memory, so it runs in the same one-CPU, two-gigabyte class as the measurement. The six build jobs recorded most closely peaked at 742 to 805 MiB.

It makes loss recoverable. The volumes are single-replica block volumes. Six of them became unreachable when their node went offline. Each was deleted, recreated under the same name and rebuilt from its seed, on the first attempt, in 2 hours 32 minutes to 3 hours 49 minutes. No backup of the index was kept, because the seed and the code reproduce it.

Build. The 500 servers were built between 2026-08-04 and 2026-09-24. The first 16 were built in waves and the remaining 484 by one pool with no wave barrier, the same pool the measurement later used. Each server took between two and a half and four hours.

4 STORAGE IS ONE VOLUME PER SERVER

Each server’s index lives on its own block volume of 56 GiB, mounted by exactly one job at a time. Every job of a phase reads only its own volume and the shared network file system, so a slow volume slows one server and no other. Volumes are provisioned at a paced rate, because a burst of claims times out the provisioner. In August a new volume bound in about 31 seconds, and the six replacements of September took about ten minutes. Scans read the

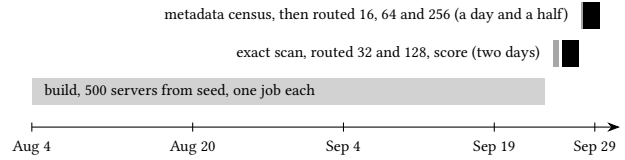


Figure 2: The run on the calendar. The index was built over seven weeks by a pool of one job per server. The measurement, the census and the second sweep took four days at the end of September, all through the same controller.

index through memory maps in blocks, so the page cache holds recently read index pages. Those pages count against the job’s memory limit, and the kernel reclaims them under it, because they are clean copies of data on the volume.

5 EVERY JOB FITS ONE CPU AND TWO GIGABYTES

Every job in the measurement requests one CPU and two gigabytes of memory. That is the smallest class the cluster offers, it is the class in which a fleet of hundreds of small jobs takes the least from other users, and the sharded index needs nothing more once three choices are made in how a scan over memory-mapped shards is written. The first is that the queries come from the cache. Deriving the 100 queries from the seed means regenerating four whole five-million-row blocks, about 640 MB each plus temporaries, so the query-cache phase writes the set once and every scan reads the file. The second is that at most two shards are open. A memory-mapped shard still materializes its row-id and tombstone arrays, 45 MB per five-million-row shard, and the sharded index holds 128 shards open by default, 5.8 GB across a full scan. The scan holds two shards open and the routed pass eight, with two worker threads. The third is that scoring proceeds in blocks of 65536 rows, which bounds the per-block temporaries to tens of megabytes.

A pilot exact-scan job at this shape finished in 1912 s with CPU averaging 884 millicores and peaking at 979, memory averaging 1242 MiB and peaking at 1469, and a process resident set of 346 MB at the end. A pilot routed-pass job finished in 36 minutes at 475 millicores average, memory averaging 1258 MiB and peaking at 1568, resident set 165 MB. The memory above the resident set is clean page cache from the mapped index, which the kernel reclaims under the limit. The routed pod was observed at the two-gigabyte limit with 224 MB anonymous and 1.9 GB file-backed and completed normally. The score job, which merges 1500 small partials, runs in the same class, and so do the metadata jobs that read the partials and the index headers.

6 A POOL OF IDEMPOTENT JOBS CARRIES THE RUN

The measurement was driven by a pool of at most twenty jobs in flight with no wave barrier, checkpointed per phase, restartable, and able to adopt jobs already on the cluster after a restart. Figure 2 places it on the calendar and Table 1 gives what it did. The rules come first because the counts are only meaningful against them.

Table 1: The pool over the first run, 2026-09-25 18:01 UTC to 2026-09-27 08:59 UTC, and the second sweep, 2026-09-27 21:37 UTC to 2026-09-29 12:56 UTC. First-run scan counts exclude 152 servers scanned in an earlier session at a larger job shape.

	first run	second sweep
submissions	928	574
completions	849	501
recycles after a failed job	75	73
storage read fault on one node	24	25
node lost during the job	13	17
node unreachable, pod stuck	13	11
container start failure	2	1
other	23	19
recycles after 45 minutes pending	26	0
submissions deferred by the controller	1	0
servers parked or given up	0	0

Rules. A job whose Kubernetes object reports success is done. A job whose object reports failure is recycled, meaning its object is deleted and, after an exponential back-off from two minutes to thirty, resubmitted, up to forty times before the server is parked and swept again later. A job submitted through the controller but not yet visible as an object is waited on for ninety polls, one a minute, before it is treated as lost. In the first run a job whose pod had not reached the running state 45 minutes after submission was recycled the same way. For the second sweep that rule was replaced by waiting, because a running job is never deleted on a namespace other groups share, and a job running longer than sixteen hours is logged rather than removed.

The deferral rule. The submission controller defers creating a job while the namespace already has several pending or the cluster is busy, and retries for up to about an hour. A driver that treats a deferred submission as a lost one resubmits, and every resubmission queues a duplicate behind the original it was meant to replace. The wait before a submission is treated as lost separates the two cases. The run needed the rule once, for the score job.

Failure classes. Two dominated, and both are ordinary on a shared cluster. One node returned a storage read fault to every job placed on it within 3 to 13 minutes of start, after its block volume had attached. Retries landed elsewhere and finished, at a cost of about 130 slot-minutes out of 9600 over the first eight hours. The second class, pods pending past 45 minutes, was benign in every instance checked, because the original pod had run and the retry found its output within minutes. That is what idempotent jobs buy. In the second sweep 63 servers needed a second attempt, 9 a third and 1 a fourth, and a storage fault at one moment ended all twenty jobs in flight at once, after which fresh submissions ran and the batch was re-issued after its back-off.

Pods that stop answering. A pod can stay in a terminating state on a node that no longer responds, or run on a node whose storage path is slow, and the job controller does not replace it on its own. Four such pods in the first run and two in the second sweep were removed by hand, and their replacements finished normally. The

Table 2: Per-server wall time in the 10^{12} run, one CPU per job, 500 servers per phase, and the CPU-hours each phase consumed. The three routed widths of the second sweep ran inside one job per server and share its start-up. Each time includes pulling the image, installing dependencies, cloning the repository and opening 400 shards from network block storage, so it is the cost of the measurement job and not of inverted-file search. Derived from the record by `paper/pvldb1t/figdata/fig_data.py`.

phase	median	mean	p10	p90	max	CPU-h
exact scan	2664 s	3008 s	1850 s	4009 s	16308 s	418
routed, 32	1064 s	1131 s	775 s	1466 s	14286 s	157
routed, 128	1335 s	1420 s	958 s	1861 s	14524 s	197
routed, 16	916 s	1071 s	660 s	1440 s	12236 s	149
routed, 64	1219 s	1366 s	912 s	1743 s	14645 s	190
routed, 256	1594 s	1719 s	1176 s	2199 s	13786 s	239

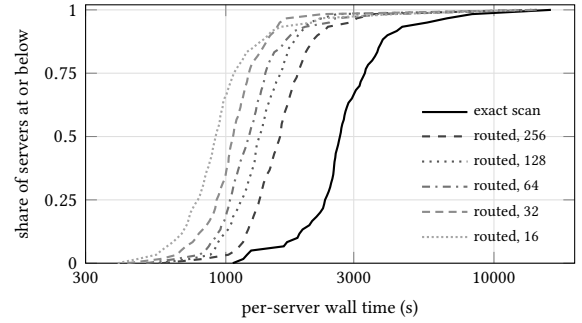


Figure 3: Distribution of per-server wall time by phase, 500 servers each, one CPU per job. The bodies are tight and the tails are a few servers on slow nodes, which the pool absorbs by waiting rather than by re-placing. Data derived from the record logs.

second sweep added a rule for pods terminating longer than thirty minutes. Across four days the manual actions were six pod removals and one resubmission of the score job.

Provenance. The job containers cloned the repository’s default branch at run time, and twelve commits appear in pod logs across the first run. The phase scripts came from a configuration map fixed for each run, and the merge and score code did not change. Pinning the clone to a commit is the one change to the descriptor for the next run.

7 THE MEASUREMENT COST 1349 CPU-HOURS

Table 2 and Figure 3 give what the measurement cost. The exact scan of two billion rows at one CPU takes 44 minutes at the median and the routed pass 15 to 27 minutes depending on width. The first run, exact scan and routed passes at 32 and 128, consumed 772 CPU-hours, and the second sweep at 16, 64 and 256 consumed 577, so the whole measurement is 1349 CPU-hours of the cluster’s

smallest job class. The routed times are not the cost of inverted-file search, and the ratio between the routed and exact rows is not a latency figure for the method. These jobs are read-bound on network block storage, spend a fixed minute or two on image pull, dependency install and clone, and open 400 shards before scoring. The 10^{11} run, on six CPUs per job, reported 14.7 and 8.5 as the same ratio, and neither number is the method's. The tails are node variance. Between 4 and 30 servers per phase took more than twice the median, the 16308 s maximum of the scan is one server on one node that also produced the slowest routed pass, and in every phase the ninetieth percentile is within 1.6 times the median.

8 WHAT WE WOULD KEEP AND WHAT WE WOULD CHANGE

We would keep the seed. It made the build free of data movement, made every lost volume a rebuild rather than a restore, and made the queries and the rerank rows of the companion paper reproducible without storing them.

We would keep the smallest class. It forced the three changes of Section 5, each of which made the job simpler, and it let the pool run twenty jobs wide beside other groups' work.

We would keep idempotency. Every recovery rule of Section 6 is safe only because a job whose output exists exits at once and a partial output is never visible.

We would change three things. The job containers cloned the repository's default branch at run time, so the code that ran moved as the branch moved, and the next run pins the clone to one commit. A rule for pods that stay in a terminating state now exists in the driver, and the next run starts with it. The query set came from four seeded shards, which concentrates the reference neighbours on few servers, and the companion paper measures a query set drawn from seeds no corpus shard uses.

9 RELATED WORK

Distributed vector databases such as Milvus [9] and its cloud-native successor [4] shard an index across nodes and merge per-node results, which is the arrangement our fleet uses without a serving layer. Billion-scale practice with Faiss [3, 5], DiskANN [8] and SPANN [2] places an index on one machine or one node's disk, and the Big ANN competitions [6, 7] fixed datasets and reporting at 10^9 rows. Recent work places a billion-scale index on distributed storage [11]. The National Research Platform [10] describes the federated cluster this run used. We found no published account of building and exactly scanning a 10^{12} -row vector index on a shared cluster in its smallest job class.

10 REPRODUCIBILITY

The repository at the availability URL carries the corpus seeds and generator, the build, scan, routed, score and metadata scripts, the pool driver with its rules, and, under the directory benchmarks/fleet/record/1t/, the pool state files, every driver log, the pilot logs and the score logs whose RESULT_JSON lines hold every per-server wall time. The cost table and the wall-time figure are produced from those logs by paper/pvldb1t/figdata/fig_data.py. Rebuilding the index from seed costs about three and a half hours per server at the pool's width.

11 CONCLUSION

A trillion-row vector index was built from a seed and measured exactly on a shared research cluster in the smallest job class it offers. The build moved no corpus bytes, a lost volume was a four-hour rebuild, and a pool of idempotent jobs with explicit rules for failure, waiting and deferral carried the four-day measurement, 1349 CPU-hours, through the failures a shared cluster produces, with six pods replaced by hand.

ACKNOWLEDGMENTS

This work used the National Research Platform's Nautilus cluster. The National Research Platform is supported in part by National Science Foundation awards CNS-1730158, ACI-1540112, ACI-1541349, OAC-1826967, OAC-2112167, CNS-2100237 and CNS-2120019. We thank its operators and community.

REFERENCES

- [1] Andrew H. Bond. 2026. Recall Does Not Move at a Trillion Rows [Experiment, Analysis & Benchmark]. Under submission to PVLDB.
- [2] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss Library. *arXiv preprint arXiv:2401.08281* (2024).
- [4] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561. <https://doi.org/10.14778/3554821.3554843>
- [5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.
- [6] Harsha Vardhan Simhadri, Martin Aumüller, Amir Ingber, Matthijs Douze, George Williams, Magdalen Dobson Manohar, Dmitry Baranchuk, Edo Liberty, Frank Liu, Ben Landrum, Mazin Karjkar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Yizhuo Chen, Weiguo Zheng, Zihao Wan, Jie Yin, and Ben Huang. 2024. Results of the Big ANN: NeurIPS'23 competition. *arXiv preprint arXiv:2409.17424* (2024).
- [7] Harsha Vardhan Simhadri, George Williams, Martin Aumüller, Matthijs Douze, Artem Babenko, Dmitry Baranchuk, Qi Chen, Lucas Hosseini, Ravishankar Krishnaswamy, Gopal Srinivasa, Suhas Jayaram Subramanya, and Jingdong Wang. 2022. Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track (Proceedings of Machine Learning Research)*, Vol. 176, 177–189.
- [8] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32.
- [9] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [10] Derek Weitzel, Ashton Graves, Sam Albin, Huijun Zhu, Frank Würthwein, Mahidhar Tatineni, Dmitry Mishin, John Graham, Elham E. Khoda, Mohammad Firas Sada, Larry Smarr, and Thomas DeFanti. 2025. The National Research Platform: Stretched, Multi-Tenant, Scientific Kubernetes Cluster. In *Practice and Experience in Advanced Research Computing (PEARC '25)*. arXiv:2505.22864.
- [11] Kun Yu, Jiabao Jin, Xiaoyao Zhong, Peng Cheng, Lei Chen, Zhitao Shen, Jingkuan Song, Hengtao Shen, and Xuemin Lin. 2025. Approximate Nearest Neighbor Search of Large Scale Vectors on Distributed Storage. *arXiv preprint arXiv:2510.17326* (2025).